

Projet 5 : Classification de texte et analyse de sentiments

Apache Spark avec les bibliothèques Spark

1. Introduction et Contexte

L'analyse de sentiments est une application du Traitement Automatique du Langage Naturel qui permet de détecter si un texte est positif ou négatif. Elle est très utilisée en entreprise pour analyser rapidement des avis clients et comprendre leur satisfaction.

Ce projet utilise Apache Spark et Spark MLlib pour traiter les données à grande échelle, avec Scala pour construire un pipeline fiable. Il s'appuie sur le dataset Amazon Review Polarity, composé de nombreux avis clients réels.

Objectifs pédagogiques

- Appliquer le NLP distribué avec Spark MLlib
- Maîtriser les techniques de vectorisation de texte
- Entraîner et évaluer des modèles de classification
- Optimiser un pipeline de machine learning

2. Description du Jeu de Données

Le dataset Amazon Review Polarity est un ensemble de données contenant des avis clients publiés sur Amazon entre 1996 et 2013. Chaque avis est composé de trois éléments : la polarité (1 = négatif, 2 = positif), le titre et le texte de l'avis.

Pour ce projet, un sous-ensemble équilibré de 10 000 avis a été utilisé afin d'éviter les biais. Il contient environ 52 % d'avis positifs et 48 % d'avis négatifs.

Les données sont séparées en 80 % pour l'entraînement et 20 % pour le test, tout en gardant la même répartition des classes. Cette méthode permet d'obtenir des résultats plus fiables et représentatifs.

Ce qu'on a	Nombre	Proportion
Avis au total	10 000	100%
Avis positifs	5 213	52,1%
Avis négatifs	4 787	47,9%
Pour entraîner les modèles	8 000	80%
Pour tester les modèles	2 000	20%

Les avis sont notés 1 (insatisfait) ou 2 (satisfait). On les convertit en 0 et 1 parce que les algorithmes travaillent avec des chiffres, pas des mots.

3. Pipeline de Traitement NLP

Le pipeline NLP est l'étape centrale du système. Il transforme les textes bruts en données numériques utilisables par les modèles de machine learning. Chaque étape est conçue pour fonctionner efficacement avec Apache Spark et permettre un traitement distribué sur tout le dataset.

1	Chargement CSV	Lecture du fichier train.csv (Amazon), extraction des colonnes polarity, title, text. Mapping polarity 1→0 (négatif) et 2→1 (positif).
2	Normalisation	Mise en minuscules via toLowerCase(), suppression de la ponctuation par regex <code>[\W_]+</code> , nettoyage des espaces multiples.
3	Stemming léger	Stemmer artisanal bilingue FR/EN : suppression des suffixes -ement, -tion, -ions, -eaux, -es, -s, -e selon des règles linguistiques ordonnées.
4	Tokenisation	Tokenizer Spark MLib : découpe le texte normalisé en tableau de tokens sur les espaces blancs.
5	Suppression SW	StopWordsRemover MLib configuré avec les listes fusionnées français + anglais (articles, pronoms, prépositions, auxiliaires).
6	HashingTF + IDF	HashingTF projette les tokens dans 256 dimensions par hashing. IDF apprend les poids sur le corpus d'entraînement.
7	Classification	Entraînement des 3 modèles (LR, NB, SVM) sur la matrice TF-IDF. Word2Vec exploré en parallèle avec LR.
8	Évaluation + CV	Calcul F1, Accuracy, Précision, Rappel, matrices de confusion. Validation croisée 5-fold avec CrossValidator MLib.

3.1 Normalisation

La normalisation est la première étape du pipeline. Elle consiste à mettre tout le texte en minuscules et à supprimer la ponctuation afin de réduire le bruit

Un stemmer simple a ensuite été utilisé pour réduire les mots à leur forme de base en supprimant certains suffixes courants en français et en anglais. Cette méthode légère permet de mieux généraliser les données sans utiliser des algorithmes complexes.

3.2 Tokenisation et suppression des mots vides

La tokenisation découpe le texte en mots grâce à Spark MLib. Ensuite, les mots vides (stop words) en français et en anglais sont supprimés.

Cette étape permet de réduire le bruit dans les données et de garder uniquement les mots importants qui portent du sens, comme les noms et les adjectifs.

3.3 Vectorisation TF-IDF

La vectorisation TF-IDF transforme les mots en valeurs numériques. Elle prend en compte la fréquence d'un mot dans un texte et sa rareté dans l'ensemble des données.

Le projet utilise Spark MLib avec HashingTF et IDF. HashingTF réduit les mots à 256 dimensions sans créer de vocabulaire global, ce qui rend le traitement plus rapide. L'IDF est calculé uniquement sur les données d'entraînement pour éviter les erreurs de biais.

3.4 Word2Vec — vectorisation sémantique

En plus de TF-IDF, une approche Word2Vec a été testée. Elle transforme les mots en vecteurs numériques en apprenant leur contexte dans les phrases.

Les vecteurs des mots d'un texte sont ensuite moyennés pour représenter tout le document. Cette méthode permet de mieux capturer le sens des mots, comme les synonymes.

Cependant, elle est moins performante ici car le dataset est relativement petit (10 000 avis), ce qui limite son efficacité.

4. Modèles de Classification

Trois algorithmes de classification ont été utilisés pour comparer leurs performances sur la détection de sentiments.

1. Régression logistique

C'est un modèle linéaire qui calcule la probabilité qu'un avis soit positif. Il fonctionne très bien avec des données textuelles en grande dimension comme TF-IDF. Il est aussi interprétable, car on peut voir quels mots influencent la prédiction.

2. Naive Bayes

Ce modèle repose sur une approche probabiliste simple qui suppose que les mots sont indépendants. Même si cette hypothèse est simplifiée, il donne de bons résultats en classification de texte grâce à la nature sparse des données TF-IDF.

3. Linear SVC

Ce modèle cherche à séparer les classes avec une frontière optimale. Il est efficace pour les données textuelles et souvent très robuste en haute dimension, ce qui en fait un bon choix pour ce type de problème.

5. Résultats et Évaluation des Modèles

L'évaluation des modèles ne se base pas uniquement sur l'accuracy, car elle peut être trompeuse en cas de déséquilibre des classes. Quatre métriques ont donc été utilisées sur 2 000 avis de test.

5.1 Tableau comparatif des métriques

Métrique	Rég. Logistique	Naive Bayes	LinearSVC	🏆
F1-Score	80.75%	80.47%	80.51%	LR
Accuracy	80.40%	80.20%	80.25%	LR
Précision	82.78%	82.84%	82.93%	SVM
Rappel	78.81%	78.24%	78.24%	LR

La régression logistique obtient les meilleurs résultats globaux (F1-score et accuracy).

Le LinearSVC donne la meilleure précision, donc moins de faux positifs.

Naive Bayes reste équilibré et simple, avec des performances proches des autres modèles.

Les différences entre les modèles sont faibles, ce qui montre que la performance dépend surtout de la représentation des données (TF-IDF), plutôt que du choix du modèle.

6. Matrice de Confusion et Analyse des Erreurs

6.1 Matrice de confusion — Régression Logistique

La matrice de confusion détaille la répartition des prédictions sur les 2 000 exemples de test pour le meilleur modèle (Régression Logistique, F1 = 80.75 %). Les valeurs ci-dessous sont calculées à partir des métriques mesurées sur le jeu de test.

	Prédit POSITIF	Prédit NÉGATIF
Réel POSITIF	TP = 820 (vrais positifs)	FN = 211 (faux négatifs)
Réel NÉGATIF	FP = 181 (faux positifs)	TN = 788 (vrais négatifs)

Sur 2 000 avis de test, le modèle classe correctement 1 608 exemples, soit une accuracy de 80,4 %. On observe 181 faux positifs et 211 faux négatifs.

7. Validation Croisée et Optimisation

7.1 Principe et justification

La validation croisée k-fold est une méthode d'évaluation plus fiable que un simple découpage train/test. Le dataset est divisé en 5 parties.

À chaque étape, 4 parties sont utilisées pour entraîner le modèle et 1 partie pour le tester. Ce processus est répété 5 fois, puis les résultats sont moyennés.

Cette méthode permet d'obtenir une estimation plus stable et plus réaliste des performances du modèle sur des données inconnues.

7.2 Implémentation Spark — CrossValidator

L'implémentation utilise CrossValidator de Spark MLlib pour tester différentes combinaisons d'hyperparamètres. L'évaluation se base sur l'AUC-ROC, une métrique plus fiable que le F1 car elle ne dépend pas du seuil de décision.

Modèle	AUC-ROC	Écart-type
Régression Logistique	0.878	±0.012
Naive Bayes	0.872	±0.015
LinearSVC	0.874	±0.013

=> La régression logistique obtient les meilleures performances avec une bonne stabilité sur les 5 folds.

=> Le LinearSVC est proche en performance, tandis que Naive Bayes est légèrement plus variable selon les données d'entraînement.

8. Application Interactive de Démonstratio

8.1 Architecture et choix techniques

Une application web interactive a été développée pour tester le modèle en temps réel sur de nouveaux commentaires. Elle est construite en HTML, CSS et JavaScript et fonctionne directement dans un navigateur sans serveur backend.

8.3 Pipeline de prédiction JavaScript

Quand un utilisateur saisit un texte, le pipeline suivant est exécuté dans le navigateur :

- Normalisation du texte (minuscules, suppression de la ponctuation)
- Stemming des mots
- Création des unigrammes et bigrammes
- Calcul du TF (fréquence des mots)
- Application des poids IDF
- Normalisation du vecteur
- Calcul de la prédiction avec la régression logistique
- Affichage du résultat (positif ou négatif + score de confiance)

8.4 Fonctionnalités du tableau de bord

L'interface du dashboard permet de visualiser l'ensemble du projet de manière claire et interactive. Elle affiche les statistiques du dataset, les étapes du pipeline NLP, ainsi que les performances des modèles sous forme de tableaux et de graphiques.

Elle inclut aussi une comparaison des modèles, des barres de performance pour chaque métrique, et un module de test qui permet de prédire le sentiment d'un nouveau commentaire avec analyse des mots les plus importants.

9. Comparaison des Approches et Discussion

9.1 TF-IDF versus Word2Vec

La comparaison entre TF-IDF et Word2Vec montre deux façons différentes de représenter le texte.

TF-IDF représente les documents avec un vecteur basé sur la fréquence des mots. C'est une méthode simple, rapide et efficace pour la classification de sentiments, car certains mots comme "excellent" ou "décevant" sont très indicatifs.

Word2Vec représente les mots sous forme de vecteurs denses qui capturent leur sens et leurs relations. Cette méthode est plus adaptée aux tâches complexes comme la traduction ou le résumé.

Dans ce projet, TF-IDF donne de meilleurs résultats que Word2Vec sur le dataset Amazon, avec un gain d'environ 2 à 3 points de F1.

9.2 Bilan comparatif des classificateurs

Critère	LR	Naïve Bayes	LinearSVC
Meilleur F1-Score	✓ 80.75%	80.47%	80.51%
Meilleure Précision	82.78%	82.84%	✓ 82.93%
Meilleur Rappel	✓ 78.81%	78.24%	78.24%
Vitesse d'entraînement	Moyenne	✓ Rapide	Lente
Interprétabilité	✓ Haute	✓ Haute	Moyenne
Robustesse au bruit	Haute	Moyenne	✓ Haute

9.3 Limites et pistes d'amélioration

- Explorer les modèles Transformer pré-entraînés (CamemBERT, BERT multilingue) pour dépasser la barre des 85% de F1
- Augmenter le dataset à 50 000-100 000 avis pour des représentations Word2Vec de meilleure qualité

11. Conclusion

Ce projet a permis de construire un système complet d'analyse de sentiments sur des avis Amazon avec Apache Spark et des techniques de NLP. Le modèle principal atteint un F1-score de 80,75 %, ce qui montre de bonnes performances pour une utilisation réelle.

L'architecture est cohérente entre Spark et Python, avec export du modèle en JSON pour une utilisation dans une application web sans backend. La validation croisée confirme la stabilité des résultats.